C2 Lakehouse, Warehouse and SQL analytics endpoint

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

Track C: Microsoft Fabric 2
C2 Lakehouse, Warehouse and SQL analytics endpoint 3
C2-L1 Lakehouse Tables and Files, and schemas 3
C2-L2 Warehouse T-SQL, and where it differs from SQL Server 7
C2-L3 Lakehouse, Warehouse or SQL analytics endpoint 11
Track C best-practice checklist 17
Appendix: solutions and answer keys 18

Track C: Microsoft Fabric

Unify data, analytics and Copilot in Fabric, and verify what Copilot produces.

C2 Lakehouse, Warehouse and SQL analytics endpoint

Roadmap: Months 16 to 18: MLOps and deployment; Current work. When to use each.

C2-L1 Lakehouse Tables and Files, and schemas

Concept

A lakehouse has two top-level areas in OneLake:

- **Tables** holds **managed Delta tables**. Fabric registers them, the SQL analytics endpoint exposes them, and Direct Lake semantic models read them.
- **Files** holds anything else: CSV, JSON, Parquet, images, raw deliveries.

Only Delta tables in Tables reach the SQL engines. A Parquet or CSV file is invisible to the SQL analytics endpoint, wherever it sits. If you put a folder that is not a valid Delta table under Tables, the lakehouse explorer shows it in an **Unidentified** area. A broken shortcut also lands there.

So the flow for a delivery is: land the raw file in **Files** (your bronze copy, as received), then read it with Spark and write a Delta table to **Tables** with `saveAsTable`.

Schemas. New lakehouses created in the portal have schemas enabled by default. Every schema-enabled lakehouse has a `dbo` schema that you cannot rename or remove. A table written without a schema goes to `dbo`. Create schemas by domain (`sales`, `hr`) to keep a growing lakehouse navigable.

In Spark SQL you reference tables with a **four-part name**: `workspace.lakehouse.schema.table`. Parts you leave out come from defaults: a bare table name means the `dbo` schema of the notebook's default lakehouse. For code that others run, name at least the lakehouse and schema, so it does not depend on which lakehouse happens to be attached.

One change to note if you learned Fabric earlier: since 5 September 2025, Fabric no longer creates a default semantic model automatically with each new lakehouse. Create semantic models on purpose (lesson C6-L1).

Why it matters

An engineer wrote cleaned survey data as Parquet into `Files/clean/respondents/`, the way they had written to cloud storage before. The Power BI team could not find the table in the SQL analytics endpoint, so they imported the Parquet into a separate dataflow, a second copy with its own refresh. Writing a Delta table to Tables with `saveAsTable` would have exposed it to every engine with no copy.

Worked example: weak code

```
❶ raw = spark.read.option("header", True).csv("Files/landing/
    respondents_w14.csv")
❷ raw.write.mode("overwrite").parquet("Files/clean/respondents")
```

1. **Line 1.** Every column read as a string, types unchecked.
2. **Line 2.** Parquet under Files: invisible to the SQL analytics endpoint and Direct Lake.

Worked example: strong code

```
 1 from pyspark.sql import DataFrame, SparkSession
 2 from pyspark.sql import types as T
 3
 4 RESPONDENT_SCHEMA = T.StructType([
 5     T.StructField("respondent_id", T.StringType(), False),
 6     T.StructField("wave", T.IntegerType(), False),
 7     T.StructField("weight", T.DoubleType(), True),
 8 ])
 9
10
11 def load_delivery(spark: SparkSession, path: str, table: str) ->
    DataFrame:
12     """Read a CSV delivery from Files and append it to a managed Delta
    table in Tables."""
❶      df = spark.read.option("header",
    True).schema(RESPONDENT_SCHEMA).csv(path)
❷      df.write.mode("append").saveAsTable(table)
15     return df
16
17
❸ load_delivery(spark, "Files/landing/respondents_w14.csv",
    "lh_survey.sales.respondents")
```

1. **Line 13.** An explicit schema: types are decided, not inferred.
2. **Line 14.** A managed Delta table in Tables: visible to Spark, T-SQL and Direct Lake.
3. **Line 18.** Lakehouse and schema named, so the code does not depend on the attached default.

Raw files in Files, Delta tables in Tables, and schema-qualified names in shared code.

Recap card

- **Lakehouse Tables vs Files** Tables: managed Delta tables, visible to SQL and Direct Lake. Files: raw and non-Delta data.
- **Non-Delta folder under Tables** Shown in the Unidentified area, not queryable.
- **Lakehouse schemas** On by default for new lakehouses. dbo is the fixed default schema.
- **Spark four-part name** workspace.lakehouse.schema.table. A bare name means dbo in the default lakehouse.

Practice

Exercise 1. Multiple choice (level 1)

A Parquet file sits at `Files/exports/orders.parquet` in a lakehouse. What can the SQL analytics endpoint do with it?

- a) Query it as a table named orders.
- b) Nothing. Only Delta tables in Tables appear in the endpoint.
- c) Query it after a metadata refresh.
- d) Convert it to Delta automatically.

Why?

- r1) The endpoint exposes Delta tables only. Parquet and CSV cannot be queried through it.
- r2) Files is a private folder only the owner can see.

Exercise 2. Multiple choice (level 2)

In a notebook attached to schema-enabled lakehouse `lh_survey`, you run `spark.read.table("respondents")`. Which table does Spark read?

- a) lh_survey.dbo.respondents
- b) The first schema that has a respondents table
- c) It raises an error: schemas require a four-part name
- d) respondents in every schema, unioned

Why?

- r1) A bare table name resolves to the dbo schema of the default lakehouse.

- r2) dbo is the only schema Spark can read.

Exercise 3. Fill the gap (level 2)

Join two tables in different workspaces in one Spark SQL query.

```
SELECT e.name, d.dept_name
FROM ops_ws.hr_lakehouse._____(1).employees AS e
JOIN global_ws.corporate_lakehouse.company.departments AS d
  ON e.deptno = d.deptno
```

Exercise 4. Write the function (level 3)

Write `load_delivery(spark, path, table)`. Read the CSV at `path` with a header and the given `RESPONDENT_SCHEMA`, append it to the managed table `table` with `saveAsTable`, and return the DataFrame you read. Do not write Parquet to Files.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Module constant.
RESPONDENT_SCHEMA = ...

# Read a CSV delivery and append it to a managed Delta table.
def load_delivery(
    spark: SparkSession,
    path: str,
    table: str,
) -> DataFrame: ...
```

Starter code:

```
from pyspark.sql import types as T

RESPONDENT_SCHEMA = T.StructType([
    T.StructField("respondent_id", T.StringType(), False),
    T.StructField("wave", T.IntegerType(), False),
    T.StructField("weight", T.DoubleType(), True),
])


def load_delivery(spark, path, table):
    df = spark.read.option("header", True).csv(path)
    df.write.mode("overwrite").parquet("Files/clean/respondents")
    return df
```

Exercise 5. Architecture design (level 4)

Lay out a lakehouse for a survey programme with raw CSV deliveries, cleaned respondent data and reporting tables.

Options: Files, Tables, dbo schema, Tables, a named schema per domain.

1. Raw CSV deliveries, kept as received
2. Cleaned respondent-level data
3. Reporting tables read by Power BI

Sources

- Lakehouse Tables holds managed Delta tables and Files holds unstructured or non-Delta data; only Delta tables appear in the SQL analytics endpoint; default semantic models are no longer created automatically since 5 September 2025. (verified: https://learn.microsoft.com/en-us/fabric/data-engineering/lakehouse-overview (updated 2026-05-07))
- Non-Delta folders under Tables and broken shortcuts appear in the Unidentified area. (verified: https://learn.microsoft.com/en-us/fabric/data-engineering/navigate-lakehouse-explorer (updated 2026-05-07))
- Schemas are enabled by default for new lakehouses; dbo cannot be renamed or removed; Spark uses workspace.lakehouse.schema.table and resolves omitted parts from defaults. (verified: https://learn.microsoft.com/en-us/fabric/data-engineering/lakehouse-schemas (updated 2026-02-24))

C2-L2 Warehouse T-SQL, and where it differs from SQL Server

Concept

Fabric Warehouse speaks T-SQL, and most SQL Server habits carry over: `CREATE TABLE`, `INSERT`, `UPDATE`, `DELETE`, stored procedures, views, `TRUNCATE TABLE`, identity columns and explicit transactions. `MERGE` is generally available. Its tables are Delta tables in OneLake, so Spark and Power BI read them too.

The differences cause real bugs. Microsoft Learn, checked September 2026:

Keys are not enforced. `PRIMARY KEY` is supported only as `NONCLUSTERED NOT ENFORCED`. `UNIQUE` likewise, and `FOREIGN KEY` only `NOT ENFORCED`. The engine uses them as hints for query plans. It does **not** stop duplicates or orphans. Deduplicate and check keys yourself (lesson A3-L1 for MERGE sources, lesson A6-L1 for contracts).

Some data types are missing, with documented alternatives:

Not supported	Use
`nchar`, `nvarchar`	`char`, `varchar` (UTF-8 collation)
`money`, `smallmoney`	`decimal`
`datetime`, `smalldatetime`	`datetime2` (up to 6 fractional digits)
`tinyint`	`smallint`
`text`, `ntext`, `json`	`varchar`

`varchar(max)` is limited to 16 MB per value.

Some features are absent or in preview. Triggers are not supported. `ALTER COLUMN` and nested CTEs are in preview. Check the T-SQL surface area page before you port a procedure that relies on them.

Porting a SQL Server script is usually a type-mapping job plus one design change: replace constraint enforcement with explicit checks.

Why it matters

A team ported a customer dimension from SQL Server, where `PRIMARY KEY` had rejected duplicate loads for years. In Fabric the same key was created `NOT ENFORCED`, so a pipeline that ran twice on the same file loaded every customer twice. Sales per customer halved in the joined reports for a week before anyone traced it. A duplicate check after each load would have failed the run on the first rerun.

Worked example: weak code

```
1 CREATE TABLE dbo.dim_customer (
2     customer_id INT PRIMARY KEY,
3     name NVARCHAR(200),
4     credit_limit MONEY,
5     created_at DATETIME
6 );
```

1. **Line 2.** An enforced clustered primary key is not supported in Fabric Warehouse.
2. **Line 3.** nvarchar is not supported. Use varchar with the UTF-8 collation.
3. **Line 4.** money is not supported. Use decimal.
4. **Line 5.** datetime is not supported. Use datetime2.

Worked example: strong code

```
1 CREATE TABLE dbo.dim_customer (
2     customer_id INT NOT NULL,
3     name VARCHAR(200),
4     credit_limit DECIMAL(19, 4),
5     created_at DATETIME2(6)
```

```
);
ALTER TABLE dbo.dim_customer
    ADD CONSTRAINT pk_dim_customer PRIMARY KEY NONCLUSTERED (customer_id)
    NOT ENFORCED;

-- The key is a hint, so check it after every load.
SELECT customer_id, COUNT(*) AS n
FROM dbo.dim_customer
GROUP BY customer_id
HAVING COUNT(*) > 1;
```

1. **Line 4.** decimal replaces money, with the scale stated.
2. **Line 8.** The only supported form of primary key: a hint for the optimiser, not a guard.
3. **Line 14.** The duplicate check the key no longer performs. Any row returned fails the load.

Map the types, declare keys as NOT ENFORCED hints, and check uniqueness yourself after every load.

Recap card

- **Primary keys in Fabric Warehouse** Only PRIMARY KEY NONCLUSTERED ... NOT ENFORCED. Duplicates are not blocked.
- **nvarchar, money, datetime in Fabric Warehouse** Not supported. Use varchar, decimal, datetime2.
- **MERGE and TRUNCATE in Fabric Warehouse** Both supported. MERGE is generally available.
- **Triggers in Fabric Warehouse** Not supported. Put the logic in the loading procedure.

Practice

Exercise 1. Multiple choice (level 1)

A Fabric Warehouse table has `PRIMARY KEY NONCLUSTERED (customer_id) NOT ENFORCED`. You insert two rows with `customer_id = 7`. What happens?

- a) The second insert fails with a key violation.
- b) Both rows are stored. The key is not enforced.
- c) The second row replaces the first.
- d) The table is locked until you remove a duplicate.

Why?

- r1) NOT ENFORCED keys are optimiser hints. The engine does not check them.

- r2) Fabric deduplicates later in a background job.

Exercise 2. Fill the gap (level 2)

Add the only supported form of primary key to `dbo.dim_customer`.

```
ALTER TABLE dbo.dim_customer
    ADD CONSTRAINT pk_dim_customer PRIMARY KEY _____(1) (customer_id)
    _____(2);
```

Exercise 3. Spot the bug (level 3)

This table definition, ported from SQL Server, fails in Fabric Warehouse. Tap one failing line, then rewrite the definition with supported types.

```
CREATE TABLE dbo.fact_payment (
    payment_id BIGINT NOT NULL,
    customer_id INT NOT NULL,
    amount MONEY NOT NULL,
    note NVARCHAR(500),
    paid_at DATETIME2(6) NOT NULL
);
```

Write down the line number of the bug. Then write the fix.

Exercise 4. Write the function (level 3)

Write a query that returns every `customer_id` appearing more than once in `dbo.dim_customer`, with its count as `n`. The load job fails if it returns any row.

Shape of the answer. The reference answer follows this outline. Replace each ... with your own code.

```
SELECT ...
FROM ...
GROUP BY ...
HAVING ...
```

Starter code:

```
SELECT DISTINCT customer_id FROM dbo.dim_customer;
```

Exercise 5. Architecture design (level 4)

You port a SQL Server data mart to Fabric Warehouse. Decide how to handle each feature.

Options: NOT ENFORCED keys plus an explicit duplicate check after each load, NOT ENFORCED keys, trusting the source.

1. Primary keys that used to block duplicates
2. An audit trigger on the orders table
3. nvarchar and money columns

Sources

- Identity columns, TRUNCATE TABLE and MERGE (GA) are supported; triggers are not; ALTER COLUMN and nested CTEs are in preview. (verified: https://learn.microsoft.com/en-us/fabric/data-warehouse/tsql-surface-area (updated 2026-08-26))
- PRIMARY KEY and UNIQUE are supported only as NONCLUSTERED NOT ENFORCED, FOREIGN KEY only as NOT ENFORCED. (verified: https://learn.microsoft.com/en-us/fabric/data-warehouse/table-constraints (updated 2026-09-08))
- Unsupported types and alternatives: nchar/nvarchar to char/varchar, money to decimal, datetime to datetime2, tinyint to smallint; varchar(max) limited to 16 MB; datetime2 limited to 6 fractional digits. (verified: https://learn.microsoft.com/en-us/fabric/data-warehouse/data-types (updated 2026-06-11))

C2-L3 Lakehouse, Warehouse or SQL analytics endpoint

Concept

Fabric gives you three ways to hold and query tables. All three store data as Delta tables in OneLake. They differ in who writes the data and in which language.

Lakehouse. You write with Spark: notebooks, Spark job definitions, Dataflows Gen2 or pipelines. It holds files and Delta tables side by side. Choose it for raw files, PySpark transformations and data science. This is the closest match to your Databricks work.

SQL analytics endpoint. Every lakehouse gets one automatically. It gives a **read-only** T-SQL surface over the lakehouse's Delta tables. You can `SELECT`, and you can create views, functions and stored procedures. You cannot `INSERT`, `UPDATE` or `DELETE` table data through it. It picks up new or changed tables through a metadata sync. Microsoft documents the usual lag as under one minute, but it varies.

Warehouse. You write with T-SQL. It supports full DML and DDL, and multi-table transactions. Choose it when a team works in T-SQL, loads data with stored procedures, or needs several tables to change together or not at all.

One rule decides most cases: **who writes the data, and in which language?** Spark writers go to a lakehouse. T-SQL writers go to a warehouse. T-SQL readers of lakehouse data use the SQL analytics endpoint.

You do not have to copy data between them. A warehouse query reads lakehouse tables in the same workspace with a three-part name: `lakehouse_name.schema.table`. Direct Lake semantic models read from either.

Check the status of each feature you rely on in VERSIONS.md before you build. Fabric changes monthly.

Why it matters

A team moves a finance reconciliation from SQL Server to Fabric. They land every table in a lakehouse, because the Databricks team already uses lakehouses. The month-end stored procedure updates three tables. On the SQL analytics endpoint it fails on its first `UPDATE`, because the endpoint is read-only.

Under deadline, someone rewrites it as three separate Spark notebook writes. On one month-end the second notebook fails. Table one holds the new month and table three holds the old one, and the reconciliation report shows a balance that does not tie. A warehouse transaction would have rolled back all three tables together.

Worked example: weak code

```
1  -- Connected to the SQL analytics endpoint of lakehouse lh_finance
2  CREATE PROCEDURE dbo.close_month @period CHAR(7) AS
3  BEGIN
2      UPDATE dbo.ledger SET status = 'closed' WHERE period = @period;
5      INSERT INTO dbo.balances
6      SELECT account_id, @period, SUM(amount) FROM dbo.ledger
7      WHERE period = @period GROUP BY account_id;
3      DELETE FROM dbo.open_items WHERE period = @period;
9  END;
```

1. **Line 1.** The endpoint gives read-only access to lakehouse tables. Every write below fails.
2. **Line 4.** UPDATE, INSERT and DELETE on lakehouse tables are not supported through the endpoint.
3. **Line 8.** Three tables must change together. Without a transaction, a failure halfway leaves them inconsistent.

Worked example: strong code

```
1  -- Connected to warehouse wh_finance
2  CREATE PROCEDURE dbo.close_month @period CHAR(7) AS
3  BEGIN
2      BEGIN TRANSACTION;
5      UPDATE dbo.ledger SET status = 'closed' WHERE period = @period;
3      INSERT INTO dbo.balances (account_id, period, balance)
```

```
 7     SELECT account_id, @period, SUM(amount) FROM dbo.ledger
 8     WHERE period = @period GROUP BY account_id;
 9     DELETE FROM dbo.open_items WHERE period = @period;
10     COMMIT TRANSACTION;
11 END;
```

1. **Line 1.** The T-SQL writer works in a warehouse, which supports full DML.
2. **Line 4.** All three changes commit together or not at all. Warehouses support multi-table transactions.
3. **Line 6.** Name the target columns. An INSERT without a column list breaks when someone adds a column.

The data and the logic are the same. The item type decides whether the procedure runs at all, and whether a failure leaves the tables consistent.

Recap card

- **Lakehouse, Warehouse or endpoint: the deciding question** Who writes the data, and in which language? Spark writes to a lakehouse, T-SQL writes to a warehouse.
- **What can you do on the SQL analytics endpoint?** SELECT and create views, functions and procedures. No INSERT, UPDATE or DELETE on table data.
- **When do you need a Warehouse over a Lakehouse?** T-SQL writes, stored procedure loads, and multi-table transactions.
- **Read a lakehouse table from a warehouse in the same workspace** Use a three-part name: lakehouse.schema.table. No copy.
- **A new lakehouse table is missing from the endpoint** Metadata sync lag. Usually under a minute. Refresh or retry before dependent steps.

Practice

Exercise 1. Multiple choice (level 1)

Which statement about a lakehouse's SQL analytics endpoint is true?

- a) You can run UPDATE on lakehouse tables through it.
- b) You can create a view over lakehouse tables in it.
- c) You create it by hand after you create the lakehouse.
- d) It stores its own copy of the lakehouse tables.

Why?

- r1) Views are T-SQL objects that read data. The endpoint allows objects that read, not statements that change table data.

- r2) Views are stored inside the Delta tables.

Exercise 2. Architecture design (level 2)

Pick the Fabric item for each workload.

Options: Lakehouse, Warehouse, SQL analytics endpoint.

1. Land raw JSON survey deliveries and clean them with PySpark
2. Finance loads budgets with T-SQL stored procedures that change several tables together
3. Analysts query the cleaned lakehouse tables with T-SQL and save a few views

Exercise 3. Spot the bug (level 3)

This script runs against the SQL analytics endpoint of `lh_research`. It fails partway. Tap the failing line, then rewrite the script so it runs on the endpoint.

```
1 -- Connected to the SQL analytics endpoint of lh_research
2 CREATE VIEW dbo.v_wave_reach AS
3 SELECT wave_id, SUM(weighted_reach) / SUM(weighted_base) AS reach_pct
4 FROM dbo.nat_metrics
5 GROUP BY wave_id;
6 GO
7 UPDATE dbo.nat_metrics SET weighted_reach = 0 WHERE wave_id = 'W00';
```

Write down the line number of the bug. Then write the fix.

Exercise 4. Write the function (level 3)

On the SQL analytics endpoint, write a view `dbo.v_wave_reach` with one row per `wave_id` and a column `reach_pct`: the sum of `weighted_reach` divided by the sum of `weighted_base` from `dbo.nat_metrics`. A wave with a zero base must return NULL, not a divide-by-zero error.

What you need to know.

`NULLIF(a, b)` returns NULL when `a` equals `b`, and `a` otherwise. Any arithmetic with NULL gives NULL, so dividing by `NULLIF(x, 0)` turns a zero divisor into a NULL result instead of an error:

```
1 SELECT 10 / NULLIF(0, 0);   -- NULL
2 SELECT 10 / NULLIF(5, 0);   -- 2
```

`CREATE VIEW name AS SELECT ...` saves a query under a name. Reading the view runs the query.

Shape of the answer. The reference answer follows this outline. Replace each ... with your own code.

```sql
CREATE VIEW ...
SELECT
    ...
FROM ...
GROUP BY ...
```

Starter code:

```sql
CREATE VIEW dbo.v_wave_reach AS
```

Exercise 5. Fill the gap (level 4)

You are connected to warehouse `wh_finance`. Lakehouse `lh_research` sits in the same workspace. Complete the query so it reads the lakehouse table without copying it.

```sql
INSERT INTO dbo.wave_summary (wave_id, weighted_base)
SELECT wave_id, SUM(weighted_base)
FROM _____(1).dbo.nat_metrics
GROUP BY wave_id;
```

Exercise 6. Multiple choice (level 5)

A pipeline runs a Spark notebook that creates a new gold table in `lh_research`. The next pipeline step immediately queries the table through the SQL analytics endpoint and fails with "Invalid object name". An hour later the same query works. What is the most likely cause?

- a) The notebook wrote the table in a format the endpoint cannot read.
- b) The endpoint had not yet synced the new table's metadata.
- c) The endpoint needs a manual refresh every time a table changes.
- d) The pipeline's identity lacks permission on new tables for one hour.

Why?

- r1) The endpoint discovers new Delta tables through an automatic sync that usually takes under a minute but has no fixed limit.
- r2) The endpoint copies data from the lakehouse on a schedule.

Practical: Lakehouse, endpoint and warehouse side by side

Platform: Fabric trial workspace. Run this in a Fabric trial workspace. You create a lakehouse and a warehouse, write one table with Spark, read it through the endpoint, and see a write fail there.

1. In your trial workspace, create a lakehouse named `lh_research`.
2. Open a new notebook attached to `lh_research`. Run:
   `spark.createDataFrame([("W01", 120.0, 400.0), ("W02", 0.0, 0.0)],`

`"wave_id STRING, weighted_reach DOUBLE, weighted_base DOUBLE").write.mode("overwrite").saveAsTable("nat_metrics")`.

3. Switch the lakehouse view to its SQL analytics endpoint. Wait for `nat_metrics` to appear. Note how long it takes.
4. Open a new SQL query on the endpoint and create the `dbo.v_wave_reach` view from exercise 4. Query it. Wave W02 returns NULL.
5. In the same query window, run `UPDATE dbo.nat_metrics SET weighted_reach = 0 WHERE wave_id = 'W01';` and read the error message.
6. Create a warehouse named `wh_finance`. In a new query, run `SELECT * FROM lh_research.dbo.nat_metrics;`. It returns the rows without a copy.
7. Write down what you saw in each step. Compare it with the recap card.

Sources

- The SQL analytics endpoint is read-only for Delta table data. No INSERT, UPDATE or DELETE. (verified: https://learn.microsoft.com/en-us/fabric/data-engineering/lakehouse-sql-analytics-endpoint (updated 2026-05-19))
- You can create views, functions and stored procedures in the SQL analytics endpoint. (verified: https://learn.microsoft.com/en-us/fabric/data-engineering/lakehouse-sql-analytics-endpoint (updated 2026-05-19))
- Fabric Warehouse supports explicit multi-table transactions with BEGIN TRAN, COMMIT TRAN and ROLLBACK TRAN. (verified: https://learn.microsoft.com/en-us/fabric/data-warehouse/transactions (updated 2026-06-03))
- Metadata sync runs in the background, and you can refresh it from the portal, the REST API or sys.sp_dw_refresh_ext_table. (verified: https://learn.microsoft.com/en-us/fabric/data-engineering/sql-analytics-endpoint-metadata-sync (updated 2026-05-29))
- A warehouse reads lakehouse tables in the same workspace with three-part names, and transactions can span them. (verified: https://learn.microsoft.com/en-us/fabric/data-warehouse/query-warehouse and .../transactions)
- Under normal conditions the lag between a lakehouse and its SQL analytics endpoint is under one minute; the background sync stops after 15 minutes without query activity. (verified: https://learn.microsoft.com/en-us/fabric/data-engineering/sql-analytics-endpoint-performance (updated 2026-08-07))

Track C best-practice checklist

☐ Check the status (GA or preview) of every Fabric feature before you depend on it.

☐ One copy of each dataset. Use shortcuts or mirroring before you copy.

☐ Lakehouse for Spark and files. Warehouse for T-SQL writes and multi-table transactions.

☐ The SQL analytics endpoint is read-only for table data. Write through Spark or a Warehouse.

☐ Semantic models use star schemas. DAX measures state their filter context.

☐ Direct Lake models are checked for fallback to DirectQuery.

☐ Every Copilot output is read, run on known data and compared against a second method.

☐ Workspaces map to domains. Access uses roles and OneLake security, not item sharing.

☐ Items live in Git. Changes move through deployment pipelines.

☐ Capacity use is monitored, and heavy jobs are scheduled off peak.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

C2-L1 Lakehouse Tables and Files, and schemas

C2-L1 exercise 1

Hints. (1) Which format does the endpoint read? (2) Which folder holds managed tables? (3) Delta, in Tables.

Answer. b) Nothing. Only Delta tables in Tables appear in the endpoint. Because: r1) The endpoint exposes Delta tables only. Parquet and CSV cannot be queried through it.

Only Delta tables reach the SQL engines. Files is for raw and non-Delta data.

Common wrong answers.

- `a`: Files is not scanned for tables. Write a Delta table to Tables with saveAsTable.
- `c`: Refreshing metadata picks up Delta tables. It never turns a Parquet file into a table.
- `d`: Fabric does not convert it. You write the Delta table yourself.
- `b|r2`: Right answer, wrong reason. Files is visible. It is just not tables.

C2-L1 exercise 2

Hints. (1) What fills in the missing parts of the name? (2) The default lakehouse and the default schema. (3) dbo.

Answer. a) lh_survey.dbo.respondents Because: r1) A bare table name resolves to the dbo schema of the default lakehouse.

`respondents` resolves to `lh_survey.dbo.respondents`. If the table lives in `sales`, name the schema.

Common wrong answers.

- `b`: Spark does not search schemas. Missing parts come from defaults: default lakehouse, dbo schema.
- `c`: Omitted parts are allowed and filled from defaults.
- `d`: Nothing is unioned. One table is resolved.
- `a|r2`: Right answer, wrong reason. Spark reads any schema you name. dbo is only the default.

C2-L1 exercise 3

Hints. (1) The four-part name is workspace.lakehouse.schema.table. (2) The third part is a schema. (3) The employees table lives in schema hrm in Microsoft's example, or dbo by default.

Answer.

```
SELECT e.name, d.dept_name
FROM ops_ws.hr_lakehouse.hrm.employees AS e
JOIN global_ws.corporate_lakehouse.company.departments AS d
  ON e.deptno = d.deptno
```

`workspace.lakehouse.schema.table` reaches any table you have permission to read, across lakehouses and workspaces.

Common wrong answers.

- g1 = `Tables`: Tables is a OneLake folder, not a schema. The third part is the schema name, such as dbo or hrm.

C2-L1 exercise 4

Hints. (1) Read with .schema(RESPONDENT_SCHEMA) and the header option. (2) Write with .write.mode("append").saveAsTable(table). (3) Return the DataFrame.

Answer.

```
from pyspark.sql import DataFrame, SparkSession
from pyspark.sql import types as T

RESPONDENT_SCHEMA = T.StructType([
    T.StructField("respondent_id", T.StringType(), False),
    T.StructField("wave", T.IntegerType(), False),
    T.StructField("weight", T.DoubleType(), True),
])


def load_delivery(spark: SparkSession, path: str, table: str) -> DataFrame:
    """Read a CSV delivery and append it to a managed Delta table.

    Args:
        spark: Active session.
        path: CSV path, usually under Files/landing.
        table: Target table, ideally lakehouse.schema.table.

    Returns:
        The DataFrame that was read.
    """
    df = spark.read.option("header", True).schema(RESPONDENT_SCHEMA).csv(path)
```

```
    df.write.mode("append").saveAsTable(table)
    return df
```

A managed Delta table in Tables serves Spark, the SQL analytics endpoint and Direct Lake from one copy.

C2-L1 exercise 5

Hints. (1) Which area holds non-Delta files? (2) Which area do SQL and Power BI read? (3) Schemas organise tables by domain.

Answer.

- Raw deliveries in Files, as received.
- Cleaned and reporting data as Delta tables in Tables, under schemas such as survey_silver and survey_gold.

Files for raw, Tables for Delta, and schemas to keep the tables organised.

Common wrong answers.

- raw = `tables_dbo`: CSV files under Tables are not Delta tables. They end up in the Unidentified area.
- gold = `files`: Power BI's Direct Lake and the SQL endpoint read Delta tables in Tables, not files.
- clean = `tables_dbo`: It works, but everything lands in one flat schema. Named schemas keep domains apart as the lakehouse grows.

C2-L2 Warehouse T-SQL, and where it differs from SQL Server

C2-L2 exercise 1

Hints. (1) What does NOT ENFORCED mean? (2) Does the engine check the key on insert? (3) No.

Answer. b) Both rows are stored. The key is not enforced. Because: r1) NOT ENFORCED keys are optimiser hints. The engine does not check them.

Both rows land. Check uniqueness after each load with a GROUP BY ... HAVING COUNT(*) > 1 query.

Common wrong answers.

- `a`: That is SQL Server behaviour for enforced keys. Fabric Warehouse keys are NOT ENFORCED.
- `c`: INSERT never replaces rows. You would need MERGE for upsert behaviour.

- `d`: Nothing locks. Both rows are simply stored.
- `b|r2`: Right answer, wrong reason. No background job removes duplicates. You must check.

C2-L2 exercise 2

Hints. (1) Not clustered. (2) Not checked by the engine. (3) `NONCLUSTERED` and `NOT ENFORCED`.

Answer.

```
1 ALTER TABLE dbo.dim_customer
2     ADD CONSTRAINT pk_dim_customer PRIMARY KEY NONCLUSTERED (customer_id)
    NOT ENFORCED;
```

`PRIMARY KEY NONCLUSTERED (...) NOT ENFORCED` is the only accepted form.

Common wrong answers.

- g1 = `CLUSTERED`: Fabric Warehouse supports primary keys only as NONCLUSTERED.

C2-L2 exercise 3

Hints. (1) Two SQL Server types have no Fabric Warehouse equivalent. (2) One for money, one for Unicode text. (3) decimal and varchar replace them.

Answer. The bug is on line 4, 5.

Fixed version:

```
1 CREATE TABLE dbo.fact_payment (
2     payment_id BIGINT NOT NULL,
3     customer_id INT NOT NULL,
4     amount DECIMAL(19, 4) NOT NULL,
5     note VARCHAR(500),
6     paid_at DATETIME2(6) NOT NULL
7 );
```

`money` and `nvarchar` are not supported. Use `DECIMAL(19, 4)` and `VARCHAR(500)`. Varchar uses a UTF-8 collation, so it stores Unicode text.

Common wrong answers.

- Line 6: datetime2 with up to 6 fractional digits is supported.
- Line 2: BIGINT is supported.

C2-L2 exercise 4

Hints. (1) Count rows per customer_id. (2) Filter groups, not rows. (3) HAVING COUNT(*) > 1.

Answer.

```sql
SELECT customer_id, COUNT(*) AS n
FROM dbo.dim_customer
GROUP BY customer_id
HAVING COUNT(*) > 1;
```

With keys unenforced, this query is the guard. Any row it returns means a duplicate got in.

C2-L2 exercise 5

Hints. (1) What no longer enforces uniqueness? (2) Which feature is not supported at all? (3) Use the documented alternatives.

Answer.

- Keys become hints, so add a duplicate check that fails the load.
- Triggers are not supported. Do the audit write explicitly in the loading procedure.
- Map each type to its documented alternative, keeping precision and scale.

A port succeeds when every lost guarantee is replaced by an explicit step.

Common wrong answers.

- pk = `trust`: Sources resend files and pipelines rerun. Without a check, duplicates load silently.
- trigger = `keep`: Fabric Warehouse does not support triggers. The CREATE TRIGGER statement fails.
- types = `string`: Strings lose numeric semantics and cost storage. Use decimal for amounts.

C2-L3 Lakehouse, Warehouse or SQL analytics endpoint

C2-L3 exercise 1

Hints. (1) The endpoint is described as read-only. Read-only for what? (2) It blocks changes to table data, not every kind of object. (3) Views, functions and stored procedures that read are allowed.

Answer. b) You can create a view over lakehouse tables in it. Because: r1) Views are T-SQL objects that read data. The endpoint allows objects that read, not statements that change table data.

The endpoint is read-only for table data, but you can create views, functions and stored procedures in it. Fabric creates it with the lakehouse, and it reads the same Delta tables in OneLake.

Common wrong answers.

- `a`: The endpoint is read-only for table data. Write with Spark in the lakehouse, or move the table to a warehouse.
- `c`: Fabric creates the endpoint automatically with every lakehouse.
- `d`: It reads the same Delta tables in OneLake. There is no second copy. It syncs metadata, not data.
- `b|r2`: Right choice, wrong reason. A view is a T-SQL object in the endpoint. The Delta tables stay unchanged.

C2-L3 exercise 2

Hints. (1) Ask who writes the data, and in which language. (2) Spark writers and files: one item. T-SQL writers: another. (3) T-SQL readers of lakehouse data need no new item at all.

Answer.

- Raw files and PySpark: **Lakehouse**. Spark writes, and files sit next to tables.
- T-SQL writes across several tables: **Warehouse**, for DML and multi-table transactions.
- T-SQL reads of lakehouse data, plus views: **SQL analytics endpoint**. No copy needed.

Spark writers use a lakehouse. T-SQL writers use a warehouse. T-SQL readers of lakehouse data use the endpoint that comes with the lakehouse.

Common wrong answers.

- finance = `lakehouse`: The finance team writes with T-SQL. A lakehouse accepts writes from Spark, and its endpoint is read-only. Use a warehouse.
- finance = `endpoint`: The endpoint cannot change table data, so the stored procedures would fail on their first UPDATE or INSERT.
- raw = `warehouse`: A warehouse does not hold raw files, and the team writes with PySpark. Use a lakehouse.
- adhoc = `warehouse`: Copying the lakehouse tables into a warehouse makes a second copy to keep in sync. The endpoint reads them where they are.

C2-L3 exercise 3

Hints. (1) Which statements change table data? (2) The endpoint accepts reads and objects that read. (3) UPDATE on a lakehouse table fails through the endpoint.

Answer. The bug is on line 7.

Fixed version:

```
-- Connected to the SQL analytics endpoint of lh_research
CREATE VIEW dbo.v_wave_reach AS
SELECT wave_id, SUM(weighted_reach) / SUM(weighted_base) AS reach_pct
FROM dbo.nat_metrics
GROUP BY wave_id;
GO
-- Fix wave W00 in the lakehouse with Spark, not here.
```

Line 7 is an `UPDATE` on a lakehouse table, which the read-only endpoint rejects. Make that correction in a Spark notebook against the lakehouse, and keep only the view here.

Common wrong answers.

- Line 2: Creating a view is allowed on the endpoint. Look for a statement that changes table data.
- Line 3: The division could hit zero, but that is not what stops this script. Look for a write.

C2-L3 exercise 4

Hints. (1) Aggregate both sums per wave, then divide. (2) `NULLIF(x, 0)` returns NULL when x is 0, and dividing by NULL gives NULL. (3) `SUM(weighted_reach) / NULLIF(SUM(weighted_base), 0) AS reach_pct`, then `GROUP BY wave_id`.

Answer.

```
CREATE VIEW dbo.v_wave_reach AS
SELECT
    wave_id,
    SUM(weighted_reach) / NULLIF(SUM(weighted_base), 0) AS reach_pct
FROM dbo.nat_metrics
GROUP BY wave_id;
```

Sum both columns per wave and divide by `NULLIF(SUM(weighted_base), 0)`. A zero base becomes NULL, so the division returns NULL instead of an error.

C2-L3 exercise 5

Hints. (1) Three-part names are item.schema.table. (2) The first part names the item that holds the table. (3) The table sits in the lakehouse `lh_research`.

Answer.

```
INSERT INTO dbo.wave_summary (wave_id, weighted_base)
SELECT wave_id, SUM(weighted_base)
FROM lh_research.dbo.nat_metrics
GROUP BY wave_id;
```

A warehouse query reads a lakehouse table in the same workspace with `lh_research.dbo.nat_metrics`. The data stays in one place.

Common wrong answers.

- g1 = `wh_finance`: `wh_finance` is the warehouse you are in. The table lives in the lakehouse, so name the lakehouse.
- g1 = `OneLake`: There is no database called OneLake. The first part of a three-part name is the item name: the lakehouse.

C2-L3 exercise 6

Hints. (1) The query failed first and worked later, with no change in between. (2) How does the endpoint learn that a new table exists? (3) Look at the metadata sync between the lakehouse and its endpoint.

Answer. b) The endpoint had not yet synced the new table's metadata. Because: r1) The endpoint discovers new Delta tables through an automatic sync that usually takes under a minute but has no fixed limit.

The endpoint syncs lakehouse metadata automatically, usually in under a minute, but the lag varies. When a later step depends on a new table, add a metadata refresh step (portal, REST API) or a retry before the query. Check the current refresh options on Microsoft Learn.

Common wrong answers.

- `a`: The query worked an hour later without any change to the table, so the format was fine.
- `c`: The sync runs automatically. A manual refresh speeds it up, but it is not required every time.
- `d`: Permissions do not change on a timer. The delay points to the metadata sync.
- `b|r2`: Right choice, wrong reason. The endpoint does not copy data. It syncs metadata about the Delta tables.